Gestión de bases de datos relacionales y NoSQL

Breve descripción:

El componente formativo aborda integralmente la gestión de bases de datos, incluyendo gestores relacionales y NoSQL, configuración de entornos y uso de SQL para definir, manipular y consultar datos. Profundiza en operaciones DML, consultas avanzadas, integración de información y bases NoSQL, además de seguridad, pruebas y respaldo para garantizar integridad, disponibilidad y confiabilidad de los datos.

Septiembre 2026

Tabla de contenido

Introducción 4

1. Gestores de bases de datos y configuración del entorno 6

1.1. Gestores de bases de datos relacionales y no relacionales 7

1.2. Gestores de bases de datos transaccionales 9

1.3. Instalación y configuración de gestores de bases de datos 11

2. Lenguaje estructurado de consulta (SQL) 16

2.1. Características, componentes y estándares de SQL..... 16

2.2. Tipos de datos, restricciones y propiedades de los datos 19

2.3. Lenguaje de definición de datos (DDL) 23

2.4. Restricciones de integridad 25

3. Manipulación y consulta de datos con SQL 28

3.1. Lenguaje de manipulación de datos (DML) 28

3.2. Operadores de comparación, aritméticos y lógicos..... 31

3.3. Funciones de cadena, matemáticas, fecha-hora y del sistema..... 34

3.4. Consultas combinadas (JOIN) 36

3.5. Sentencias de agregación 38

4. Gestión de bases de datos NoSQL 42

4.1. Conceptos de gestores NoSQL..... 42

4.2. Instalación y configuración de bases de datos NoSQL 44

4.3. Construcción de bases de datos NoSQL 46

4.4. Operaciones CRUD, consultas e índices 50

5. Seguridad, pruebas y respaldo de bases de datos 53

5.1. Lenguaje de control de datos (DCL): GRANT y REVOKE 53

5.2. Pruebas de bases de datos: `scripts` de definición, integridad de datos e integridad referencial 55

5.3. Gestión de copias de seguridad: exportación e importación de bases de datos 57

Síntesis 60

Glosario 62

Referencias bibliográficas 63

Créditos 64

Introducción

El componente formativo orientado a la gestión de bases de datos constituye un pilar fundamental dentro del desarrollo de soluciones informáticas modernas, debido a la creciente necesidad de almacenar, procesar y analizar grandes volúmenes de información de manera eficiente, segura y estructurada. En este contexto, el manejo adecuado de gestores de bases de datos, tanto relacionales como no relacionales, se convierte en una competencia esencial para el desarrollo de sistemas robustos y escalables.

En primera instancia, se abordan los fundamentos relacionados con los gestores de bases de datos y la configuración del entorno, permitiendo comprender las características, diferencias y aplicaciones de los sistemas relacionales y NoSQL. Asimismo, se profundiza en los gestores transaccionales, destacando su importancia en entornos donde la integridad y consistencia de la información son críticas. Este componente también contempla los procesos de instalación y configuración, facilitando la preparación de entornos de trabajo adecuados para la administración de datos.

Posteriormente, se desarrolla el estudio del lenguaje estructurado de consulta, conocido como SQL, el cual permite definir, manipular y consultar datos dentro de una base de datos. Se analizan sus características, componentes y estándares, así como los tipos de datos, restricciones y propiedades que garantizan la integridad de la información. De igual manera, se abordan las sentencias del lenguaje de definición de datos (DDL) y las restricciones de integridad, fundamentales para el diseño de estructuras coherentes y confiables.

En continuidad, se profundiza en la manipulación y consulta de datos mediante el uso del lenguaje de manipulación de datos (DML), integrando operaciones como inserción, actualización, eliminación y consulta. Se incluyen también operadores, funciones y consultas combinadas que permiten realizar análisis complejos sobre la información almacenada, optimizando el acceso y procesamiento de los datos.

Adicionalmente, se incorpora el estudio de las bases de datos NoSQL, destacando su relevancia en escenarios donde se requiere flexibilidad, escalabilidad y manejo de datos no estructurados. Se abordan conceptos, procesos de instalación, construcción de bases de datos y operaciones CRUD, así como el uso de índices para mejorar el rendimiento.

Finalmente, el componente integra aspectos relacionados con la seguridad, pruebas y respaldo de bases de datos, abordando el lenguaje de control de datos (DCL), las estrategias de validación mediante pruebas y la gestión de copias de seguridad. Estos elementos garantizan la protección, disponibilidad e integridad de la información, consolidando buenas prácticas en la administración de sistemas de datos.

Este proceso formativo busca desarrollar competencias técnicas y analíticas que le permitan diseñar, implementar y administrar bases de datos de manera eficiente, contribuyendo al desarrollo de soluciones informáticas alineadas con las necesidades del entorno tecnológico actual.

1. Gestores de bases de datos y configuración del entorno

Los gestores de bases de datos constituyen herramientas fundamentales en el desarrollo de sistemas de información, ya que permiten administrar, organizar y controlar grandes volúmenes de datos de manera estructurada. Su uso es esencial en aplicaciones modernas, donde la disponibilidad, integridad y seguridad de la información representan factores críticos para el funcionamiento de las organizaciones.

Un gestor de base de datos, también conocido como DBMS (Database Management System), es un conjunto de programas que permiten crear, modificar, consultar y administrar bases de datos. Estos sistemas facilitan la interacción entre los usuarios y los datos, proporcionando mecanismos eficientes para su almacenamiento, recuperación y manipulación.

En la actualidad, existen diferentes tipos de gestores de bases de datos, los cuales se clasifican según su modelo de datos, su arquitectura y su propósito. Entre los más destacados se encuentran los gestores relacionales, los no relacionales y los transaccionales, cada uno con características específicas que los hacen adecuados para distintos contextos de aplicación.

Asimismo, la correcta configuración del entorno de trabajo es un aspecto clave en la gestión de bases de datos, ya que garantiza el funcionamiento adecuado del sistema, optimiza el rendimiento y facilita la administración de los recursos. Este proceso incluye la instalación del gestor, la configuración de parámetros, la creación de usuarios y la definición de políticas de seguridad.

La elección del gestor de base de datos debe realizarse considerando factores como el tipo de aplicación, el volumen de datos, la escalabilidad requerida y el nivel de seguridad necesario.

1.1. Gestores de bases de datos relacionales y no relacionales

Los gestores de bases de datos pueden clasificarse principalmente en dos grandes categorías: relacionales y no relacionales. Esta clasificación responde a la forma en que los datos son estructurados, almacenados y gestionados dentro del sistema.

A. Gestores de bases de datos relacionales

Se basan en el modelo relacional, el cual organiza la información en tablas compuestas por filas y columnas. Cada tabla representa una entidad, mientras que las relaciones entre ellas se establecen mediante claves primarias y foráneas. Este enfoque permite garantizar la integridad y consistencia de los datos, siendo ampliamente utilizado en aplicaciones empresariales.

Entre las principales características de los sistemas relacionales se destacan:

- **Estructura basada en tablas**

La información se organiza en tablas formadas por filas y columnas, donde cada tabla representa una entidad del sistema y facilita la organización lógica de los datos.

- **Uso del lenguaje SQL para la manipulación de datos**

Se utiliza SQL como lenguaje estándar para consultar, insertar, actualizar y eliminar información, permitiendo una interacción eficiente y controlada con las tablas.

- **Integridad referencial**

Se mantiene mediante claves primarias y foráneas, asegurando que las relaciones entre tablas sean coherentes y que no existan datos huérfanos o inconsistentes.

- **Transacciones seguras**

Permiten ejecutar operaciones de manera confiable, garantizando que los cambios se realicen completamente o se reviertan ante errores, preservando la estabilidad del sistema.

- **Alta consistencia de datos**

Asegura que la información almacenada sea precisa y coherente en todo momento, incluso cuando es utilizada simultáneamente por múltiples usuarios o aplicaciones.

B. Gestores de bases de datos no relacionales

Conocidos como NoSQL, surgen como una alternativa para manejar grandes volúmenes de datos no estructurados o semiestructurados. Estos sistemas no utilizan tablas tradicionales, sino que emplean modelos como documentos, grafos, clave-valor o columnas.

Las principales características de los sistemas NoSQL incluyen:

- **Alta escalabilidad horizontal**

Permite aumentar la capacidad del sistema agregando más servidores o nodos, facilitando el manejo de grandes volúmenes de datos sin afectar el rendimiento.

- **Flexibilidad en la estructura de datos**

Admite datos no estructurados o semiestructurados, permitiendo que los registros tengan diferentes formatos y evolucionen sin necesidad de redefinir la base de datos.

- **Alto rendimiento en grandes volúmenes de información**

Optimiza el acceso y procesamiento de grandes cantidades de datos, ofreciendo respuestas rápidas incluso con cargas intensivas y alto número de consultas.

- **Ausencia de esquemas rígidos**

No exige una estructura fija previa, lo que posibilita almacenar información con diferentes atributos y adaptarse fácilmente a cambios en los requerimientos.

- **Adaptabilidad a entornos distribuidos**

Está diseñado para funcionar de forma eficiente en arquitecturas distribuidas, asegurando disponibilidad, tolerancia a fallos y acceso concurrente a los datos.

Los sistemas relacionales priorizan la consistencia, mientras que los NoSQL priorizan la escalabilidad y el rendimiento.

1.2. Gestores de bases de datos transaccionales

Los gestores de bases de datos transaccionales están diseñados para garantizar la ejecución segura y confiable de operaciones que involucran múltiples pasos. Estos sistemas se caracterizan por cumplir con las propiedades ACID, las cuales aseguran la integridad de las transacciones.

Las propiedades ACID son fundamentales para comprender el comportamiento de estos sistemas y representan lo siguiente:

- **Atomicidad**: garantiza que una transacción se complete totalmente o no se ejecute en absoluto
- **Consistencia**: asegura que los datos permanezcan válidos antes y después de la transacción
- **Aislamiento**: evita interferencias entre transacciones concurrentes
- **Durabilidad**: garantiza que los datos persistan incluso ante fallos del sistema

Un fallo en la gestión transaccional puede generar inconsistencias graves en la información, afectando directamente la operación del sistema.

Para comprender las diferencias y aplicaciones de los gestores de bases de datos previamente relacionados, resulta útil realizar una comparación sintética de sus características más relevantes. La siguiente tabla resume los aspectos clave de los gestores relacionales, no relacionales y transaccionales, destacando su estructura, enfoque funcional y principales contextos de uso:

Tabla 1. Comparación de los principales tipos de gestores de bases de datos

Tipo de gestor	Estructura y modelo de datos	Características clave	Enfoque principal	Ejemplo de uso
Gestores de bases de datos relacionales	Modelo relacional basado en tablas (filas y columnas) con claves	Uso de SQL, integridad referencial, transacciones	Organización estructurada y control riguroso de la información.	Sistemas administrativos y empresariales.

Tipo de gestor	Estructura y modelo de datos	Características clave	Enfoque principal	Ejemplo de uso
	primarias y foráneas.	seguras, alta consistencia de datos.		
Gestores de bases de datos no relacionales (NoSQL)	Modelos flexibles: documentos, grafos, clave-valor o columnas.	Alta escalabilidad horizontal, flexibilidad de datos, ausencia de esquemas rígidos, alto rendimiento.	Manejo de grandes volúmenes de datos no estructurados o distribuidos.	Plataformas web, redes sociales, análisis masivo de datos.
Gestores de bases de datos transaccionales	Estructuras orientadas a operaciones secuenciales y controladas.	Cumplimiento de propiedades ACID: atomicidad, consistencia, aislamiento y durabilidad.	Ejecución segura y confiable de transacciones críticas.	Sistemas bancarios, financieros y comerciales.

1.3. Instalación y configuración de gestores de bases de datos

La instalación y configuración de un gestor de bases de datos constituye un proceso fundamental para garantizar su correcto funcionamiento dentro de un entorno tecnológico. Este proceso permite preparar el sistema para almacenar, gestionar y proteger la información de manera eficiente.

Por lo anterior, para conocer este proceso de instalación y configuración, se recomienda acceder al siguiente video:

Video 1. Instalación y configuración de gestores de bases de datos



Enlace de reproducción del video

Transcripción del video: Instalación y configuración de gestores de bases de datos
Antes de iniciar el uso de un gestor de bases de datos, es fundamental realizar correctamente su instalación y configuración inicial. En el siguiente procedimiento se describen, paso a paso, las acciones necesarias para instalar el software, definir los parámetros básicos del sistema y configurar el entorno de trabajo, garantizando un funcionamiento seguro, eficiente y acorde con las necesidades institucionales: Paso 1. Descarga del software

Acceder al sitio oficial del gestor de bases de datos y descargar la versión compatible con el sistema operativo. Verificar los requisitos mínimos antes de iniciar la instalación.

Paso 2. Ejecución del asistente de instalación

Iniciar el asistente de instalación y seleccionar los componentes necesarios. Durante este proceso, definir la ruta de instalación, configurar el servidor, establecer los puertos de conexión y seleccionar el tipo de autenticación requerido.

Paso 3. Finalización de la instalación

Completar el proceso siguiendo las indicaciones del asistente y verificar que el servicio del gestor quede activo y funcionando correctamente.

Paso 4. Configuración inicial del entorno

Crear los usuarios y roles necesarios según las funciones administrativas y operativas. Asignar los permisos correspondientes para garantizar un acceso controlado a la información.

Paso 5. Implementación de políticas de seguridad

Configurar las políticas de seguridad, incluyendo contraseñas, autenticación y restricciones de acceso, con el fin de proteger la base de datos y prevenir usos no autorizados.

Paso 6. Ajuste de parámetros de rendimiento

Realizar la configuración de parámetros de rendimiento según la carga esperada del sistema, optimizando el uso de recursos y asegurando un funcionamiento eficiente y estable.

Recuerde… Una configuración inadecuada puede afectar el rendimiento, la seguridad y la disponibilidad de la base de datos.

Además del procedimiento anterior, es recomendable implementar buenas prácticas como:

- **Realizar actualizaciones periódicas**

Consiste en mantener el gestor de bases de datos y sus componentes en versiones actualizadas, corrigiendo vulnerabilidades, mejorando el rendimiento y asegurando compatibilidad con nuevas funcionalidades.

- **Configurar copias de seguridad automáticas**

Implica programar respaldos regulares de la información para prevenir pérdidas de datos ante fallos del sistema, errores humanos o incidentes de seguridad.

- **Monitorear el rendimiento del sistema**

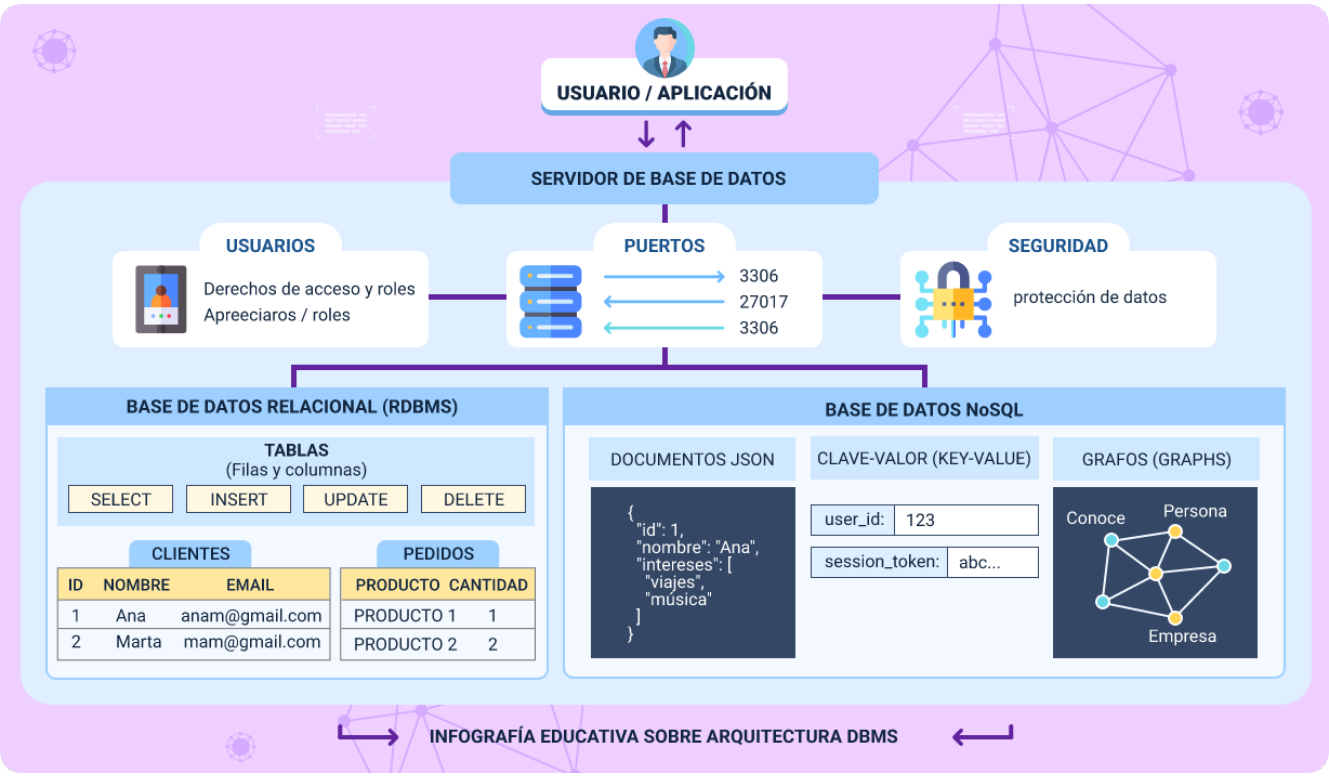
Permite supervisar el uso de recursos, tiempos de respuesta y carga del servidor, facilitando la detección temprana de problemas y la optimización del desempeño.

- **Aplicar políticas de acceso restringido**

Busca limitar el acceso a la base de datos según roles y responsabilidades, garantizando que solo usuarios autorizados puedan consultar o modificar la información.

Basado en lo explicado en esta temática, la siguiente imagen presenta una visión general de la arquitectura de un sistema de gestión de bases de datos, mostrando la interacción entre usuarios o aplicaciones y el servidor de base de datos. En ella se comparan los enfoques relacional (RDBMS) y NoSQL, junto con elementos clave como usuarios, roles, puertos de conexión, seguridad y tipos de almacenamiento de la información:

Figura 1. Diagrama de casos de uso - Sistema de biblioteca



2. Lenguaje estructurado de consulta (SQL)

El lenguaje estructurado de consulta, conocido como SQL (Structured Query Language), constituye el estándar más utilizado para la gestión y manipulación de bases de datos relacionales. Su importancia radica en la capacidad de permitir la interacción directa con los datos, facilitando su definición, consulta, actualización y control dentro de un sistema de gestión de bases de datos.

Desde una perspectiva funcional, SQL no es únicamente un lenguaje de consulta, sino un conjunto integral de instrucciones que permiten administrar completamente una base de datos. Su diseño declarativo permite que el usuario indique qué desea obtener sin necesidad de especificar cómo hacerlo, delegando esta responsabilidad al motor del gestor de base de datos.

El uso de SQL se extiende a múltiples entornos, desde aplicaciones empresariales hasta sistemas de análisis de datos, lo que lo convierte en una herramienta esencial dentro del desarrollo de software y la gestión de información.

El dominio de SQL permite no solo consultar datos, sino también diseñar estructuras robustas y garantizar la integridad de la información.

2.1. Características, componentes y estándares de SQL

El lenguaje SQL se caracteriza por ser declarativo, estructurado y orientado a la manipulación de datos en bases de datos relacionales. Su diseño facilita la interacción con grandes volúmenes de información de manera eficiente y organizada.

Entre sus principales características se destacan:

a) **Lenguaje declarativo**: el usuario define el resultado esperado sin especificar el procedimiento.
b) **Portabilidad**: puede utilizarse en diferentes gestores de bases de datos.
c) **Estandarización**: basado en normas internacionales definidas por ANSI e ISO.
d) **Capacidad de integración**: se puede combinar con otros lenguajes como Java, Python o PHP.
e) **Soporte para operaciones complejas**: facilita la ejecución de consultas avanzadas.

El lenguaje SQL se divide en varios componentes que permiten organizar sus funcionalidades así:

- **DDL**: lenguaje de definición de datos.
- **DML**: lenguaje de manipulación de datos.
- **DCL**: lenguaje de control de datos.
- **TCL**: lenguaje de control de transacciones.

Esta división permite estructurar el uso de SQL según el tipo de operación que se desea realizar.

Con respecto a los estándares de SQL, el lenguaje ha evolucionado a través de diversas versiones estandarizadas. Por ello, a continuación, se presenta una línea de tiempo que resume la evolución de los principales estándares de SQL, destacando los avances más relevantes incorporados en cada versión:

- **1986 – SQL-86**

Corresponde a la primera versión oficial del lenguaje SQL, estandarizada para establecer una base común en la gestión de bases de datos relacionales. Definió las instrucciones fundamentales para la creación de estructuras y la manipulación básica de datos.

- **1992 – SQL-92**

Representó una ampliación importante del estándar inicial, incorporando mejoras sustanciales en las consultas. Se fortalecieron las cláusulas de selección, filtrado y combinación de tablas, lo que permitió consultas más precisas y eficientes.

- **1999 – SQL: 1999**

Introdujo características avanzadas como el soporte para programación orientada a objetos. Esto permitió el uso de tipos de datos definidos por el usuario, procedimientos almacenados y una mayor complejidad en la lógica de las bases de datos.

- **2003 – SQL: 2003**

Añadió funciones analíticas que facilitaron el análisis de grandes volúmenes de datos. Estas funciones permitieron realizar cálculos avanzados, agregaciones y análisis de tendencias directamente desde las consultas SQL.

Estos estándares garantizan compatibilidad entre distintos sistemas gestores, aunque cada uno puede incluir extensiones propias.

2.2. Tipos de datos, restricciones y propiedades de los datos

Los tipos de datos en SQL permiten definir la naturaleza de la información que será almacenada en una tabla, asegurando coherencia y optimización en el uso del sistema.

A. Clasificación de tipos de datos

Los tipos de datos en los sistemas de bases de datos se agrupan en varias categorías con el fin de clasificar y estructurar la información de acuerdo con su naturaleza, formato y uso. Esta clasificación permite un almacenamiento eficiente, asegura la integridad de los datos y facilita su manipulación, validación y consulta dentro de los procesos de gestión de la información.

A partir de esta organización, se presentan las categorías más utilizadas en los gestores de bases de datos, junto con los tipos de datos más representativos que permiten definir valores numéricos, textuales, temporales y lógicos de forma adecuada:

- **Datos numéricos**

Se utilizan para almacenar valores cuantitativos y realizar operaciones matemáticas.

  - **INT**: guarda números enteros sin decimales.
  - **DECIMAL**: almacena números con precisión fija, ideal para valores financieros.
  - **FLOAT**: permite números con decimales de precisión variable.

- **Datos de texto**

Permiten almacenar información alfanumérica como nombres, descripciones o códigos.

  - **VARCHAR**: texto de longitud variable.
  - **CHAR:** texto de longitud fija.
  - **TEXT:** almacena cadenas de texto extensas.

- **Datos de fecha y hora**

Se emplean para registrar información temporal.

  - **DATE:** guarda únicamente la fecha.
  - **TIME:** almacena la hora.
  - **DATETIME:** combina fecha y hora en un solo valor.

- **Datos booleanos**

Se utilizan para representar valores lógicos.

  - **BOOLEAN**: admite valores verdadero o falso.

La selección incorrecta del tipo de dato puede afectar el rendimiento y la integridad de la información.

B. Restricciones de datos

Son mecanismos que se aplican a las columnas de una tabla para garantizar la validez, coherencia e integridad de la información almacenada. Su uso permite prevenir

errores, evitar inconsistencias y asegurar que los datos cumplan con las reglas definidas por el sistema y las necesidades del modelo de base de datos.

A continuación, se presentan las principales restricciones utilizadas para controlar los valores que pueden almacenarse en una columna, explicando su función dentro del diseño de bases de datos y su importancia para mantener información confiable, consistente y alineada con las reglas establecidas por el sistema:

- NOT NULL: impide valores nulos.
- UNIQUE: evita duplicados.
- PRIMARY KEY: identifica registros únicos.
- FOREIGN KEY: establece relaciones entre tablas.
- CHECK: valida condiciones específicas.

C. Propiedades de los datos

Las propiedades de los datos complementan la definición de los campos dentro de una tabla, permitiendo ajustar su comportamiento, formato y valores asociados. A través de estas propiedades se optimiza el almacenamiento, se automatizan ciertos procesos y se garantiza que la información cumpla con las características requeridas por el sistema.

Por lo anterior, se presentan las principales propiedades que pueden aplicarse a los campos de una base de datos, destacando cómo cada una contribuye a una gestión más eficiente, controlada y coherente de la información almacenada:

- **Auto incremento**

Permite que el sistema asigne automáticamente un valor numérico consecutivo a un campo, generalmente utilizado para identificar de forma única cada registro sin intervención del usuario.

- **Valor por defecto (DEFAULT)**

Define un valor que se asigna automáticamente a un campo cuando no se especifica uno al momento de insertar un registro, facilitando la consistencia de los datos.

- **Longitud máxima**

Establece el número máximo de caracteres o dígitos que puede almacenar un campo, ayudando a controlar el tamaño de la información y optimizar el uso del almacenamiento.

- **Precisión y escala**

Se aplica a datos numéricos decimales, donde la precisión indica el total de dígitos permitidos y la escala determina cuántos de ellos corresponden a la parte decimal.

Estas propiedades permiten adaptar la estructura de la base de datos a las necesidades del sistema.

2.3. Lenguaje de definición de datos (DDL)

El lenguaje de definición de datos (DDL) forma parte fundamental del lenguaje SQL y se utiliza para definir, crear y administrar la estructura de una base de datos. A través de este lenguaje es posible establecer cómo se organizan los datos, qué objetos existen y cómo se relacionan entre sí dentro del sistema.

A continuación, se presentan los principales comandos del DDL, junto con su función y ejemplos prácticos, que permiten comprender cómo crear, modificar y eliminar estructuras en una base de datos, así como las diferencias clave entre algunas de estas operaciones:

- **CREATE**

Permite crear objetos dentro de la base de datos, como bases de datos, tablas, vistas o índices. A través de este comando se define la estructura inicial de los objetos, especificando columnas, tipos de datos, restricciones y relaciones, constituyendo el punto de partida para el almacenamiento de la información.

```
CREATE TABLE clientes (
id INT PRIMARY KEY,
nombre VARCHAR(100),
edad INT
);
```

- **ALTER**

Permite modificar la estructura de objetos existentes sin necesidad de eliminarlos. Con este comando es posible agregar, cambiar o eliminar columnas, ajustar tipos de datos, establecer nuevas restricciones o actualizar configuraciones, facilitando la adaptación de la base de datos a nuevos requerimientos.

ALTER TABLE clientes ADD correo VARCHAR(100);

- **DROP**

Elimina completamente un objeto de la base de datos, junto con toda la información y estructura asociada. Este comando se utiliza cuando un objeto ya no es necesario y debe emplearse con precaución, ya que la operación es irreversible y no permite recuperación de los datos eliminados.

DROP TABLE clientes;

Nota: esta operación es irreversible.

- **TRUNCATE**

Elimina de forma inmediata todos los registros almacenados en una tabla, manteniendo intacta su estructura, columnas y restricciones. A diferencia de otras operaciones de eliminación, se ejecuta de manera rápida y eficiente, siendo útil para limpiar tablas sin afectar su definición.

TRUNCATE TABLE clientes;

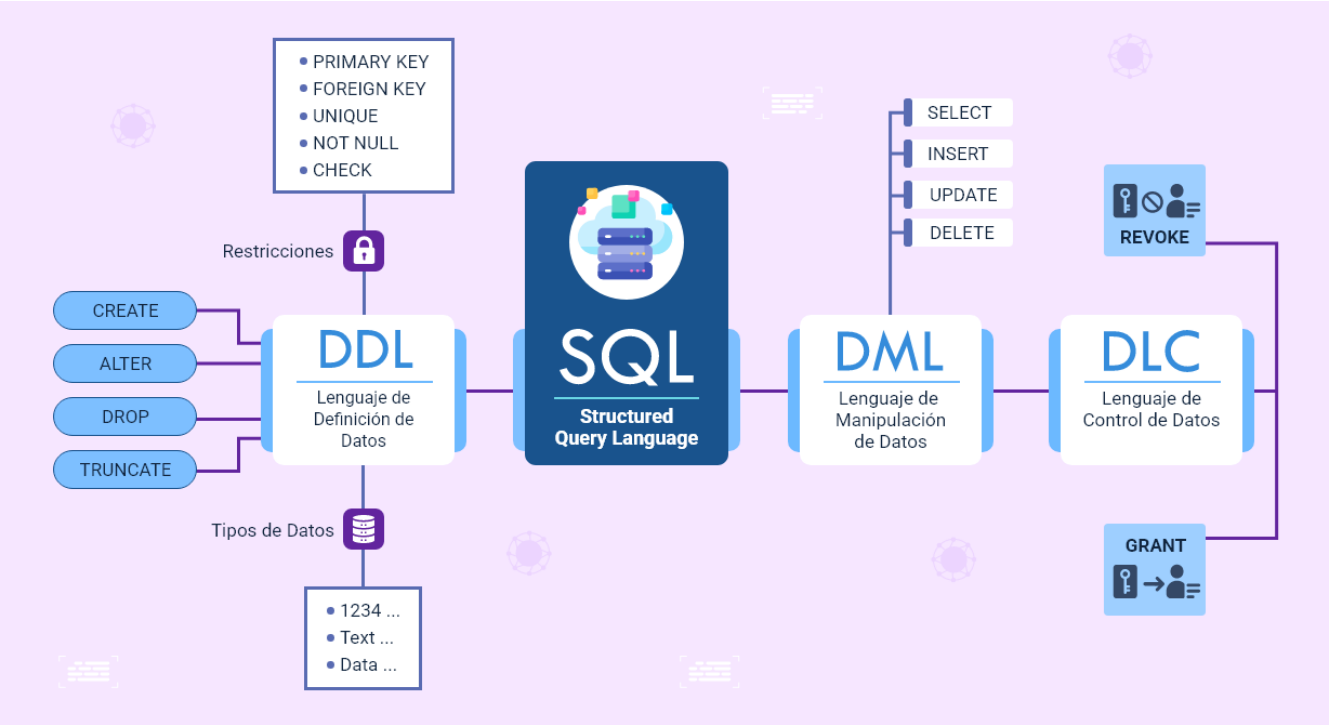
Nota:

- DELETE: elimina registros uno a uno.

- TRUNCATE: elimina todos de forma inmediata.

Como complemento, se incluye la siguiente imagen de una representación de la estructura del lenguaje SQL, mostrando su organización en distintos componentes y lenguajes especializados. En ella se identifican las funciones del DDL, DML y DCL, junto con los principales comandos, tipos de datos y restricciones, permitiendo comprender de forma integrada cómo se definen, manipulan y controlan los datos dentro de un sistema de bases de datos:

Figura 2. Estructura y componentes del lenguaje SQL



2.4. Restricciones de integridad

Las restricciones de integridad cumplen un papel esencial en el diseño y administración de bases de datos, ya que permiten asegurar que la información

almacenada sea válida, coherente y confiable a lo largo del tiempo. Estas restricciones actúan como reglas que controlan los valores que pueden registrarse en las tablas, evitando errores comunes como duplicidades, relaciones incorrectas entre datos o el ingreso de información que no cumple con las condiciones establecidas. Gracias a su aplicación, los sistemas de información mantienen un alto nivel de calidad de los datos, lo cual resulta fundamental para la toma de decisiones, los procesos operativos y el análisis confiable de la información.

Por lo anterior, se presentan los distintos enfoques de integridad de los datos, con el fin de comprender cómo cada uno contribuye al control y validación de la información dentro de una base de datos. A través de explicaciones conceptuales y ejemplos ilustrativos, se abordarán las principales formas de integridad utilizadas en SQL, permitiendo identificar su finalidad y aplicación general, sin profundizar aún en su implementación técnica detallada:

A. Integridad de unicidad

Garantiza que determinados datos no se repitan dentro de una tabla, asegurando que cada registro pueda identificarse de forma única. Se implementa principalmente mediante claves primarias y restricciones UNIQUE, las cuales impiden la existencia de valores duplicados en una o varias columnas. Esta integridad es esencial para evitar ambigüedades, mantener la identificación correcta de los registros y asegurar la confiabilidad de la información almacenada.

UNIQUE (correo)

B. Integridad referencial

Asegura que las relaciones entre tablas se mantengan de manera coherente y válida. Se establece mediante claves foráneas, las cuales obligan a que un valor en una tabla relacionada exista previamente en la tabla de referencia. De este modo, se evita la creación de registros huérfanos y se preserva la consistencia de los datos en estructuras relacionales, especialmente en bases de datos con múltiples tablas interconectadas.

FOREIGN KEY (id_cliente) REFERENCES clientes(id)

C. Integridad de dominio

Controla que los valores almacenados en una columna cumplan con condiciones específicas definidas previamente. Estas condiciones pueden incluir rangos numéricos, formatos, tipos de datos o reglas lógicas, y se aplican mediante restricciones como CHECK. La integridad de dominio permite validar la información desde el momento de su ingreso, reduciendo errores y garantizando que los datos sean coherentes con las reglas del negocio o del sistema.

CHECK (edad >= 18)

Sin restricciones de integridad, la base de datos puede volverse inconsistente e inutilizable.

El lenguaje SQL no solo permite interactuar con los datos, sino que constituye la base para el diseño, control y administración de sistemas de información. Su correcta aplicación garantiza la integridad, seguridad y eficiencia en el manejo de bases de datos, convirtiéndose en una herramienta indispensable para el desarrollo de soluciones tecnológicas modernas.

3. Manipulación y consulta de datos con SQL

La manipulación y consulta de datos constituye uno de los aspectos más importantes en la gestión de bases de datos, ya que permite interactuar directamente con la información almacenada. A través del lenguaje SQL, es posible realizar operaciones que van desde la inserción de datos hasta la ejecución de consultas complejas para el análisis de información.

Este conjunto de operaciones se agrupa dentro del lenguaje de manipulación de datos, conocido como DML (Data Manipulation Language), el cual proporciona herramientas para gestionar el contenido de las tablas sin alterar su estructura.

A diferencia del DDL, que define la estructura, el DML se enfoca en trabajar con los datos existentes, permitiendo su modificación, consulta y eliminación según las necesidades del sistema.

Una mala manipulación de datos puede generar pérdida de información o inconsistencias críticas en el sistema.

3.1. Lenguaje de manipulación de datos (DML)

Para este apartado se presentan las principales instrucciones que conforman el lenguaje de manipulación de datos, las cuales permiten ejecutar acciones específicas sobre la información almacenada en las tablas y facilitan su gestión operativa dentro de una base de datos.

A continuación, se describen los comandos más representativos del DML, junto con su función y ejemplos prácticos que ilustran su uso en escenarios habituales de manejo de datos:

A. INSERT

Permite agregar nuevos registros a una tabla, incorporando información que no existía previamente en la base de datos. Se utiliza para almacenar datos iniciales o registrar nuevos eventos, usuarios o transacciones, respetando las restricciones definidas en la estructura de la tabla.

```
INSERT INTO clientes (id, nombre, edad)
VALUES (1, 'Carlos', 30);
```

También es posible insertar múltiples registros:

```
INSERT INTO clientes (id, nombre, edad)
VALUES
(2, 'Ana', 25),
(3, 'Luis', 40);
```

B. UPDATE

Permite modificar los valores de uno o varios campos en registros ya existentes. Se emplea cuando es necesario corregir, actualizar o ajustar información almacenada, y debe utilizarse junto con condiciones que aseguren que solo se alteren los registros deseados.

UPDATE clientes

SET edad = 31

WHERE id = 1;

Nota: siempre usar WHERE, de lo contrario se actualizan todos los registros.

C. DELETE

Permite eliminar registros de una tabla cuando ya no son necesarios o deben ser descartados. Su uso implica la eliminación permanente de los datos seleccionados, por lo que es fundamental aplicarlo de manera controlada para evitar pérdidas de información no intencionadas.

DELETE FROM clientes

WHERE id = 3;

D. MERGE

Permite combinar operaciones de inserción y actualización en una sola instrucción. Es especialmente útil en procesos de sincronización de datos, ya que evalúa la existencia previa de los registros y decide si deben actualizarse o insertarse según una condición definida.

MERGE INTO clientes AS destino

USING nuevos_clientes AS origen

ON destino.id = origen.id

WHEN MATCHED THEN

UPDATE SET destino.nombre = origen.nombre

WHEN NOT MATCHED THEN

INSERT (id, nombre) VALUES (origen.id, origen.nombre);

E. SELECT

Es la instrucción más utilizada del DML y permite consultar, filtrar y recuperar información almacenada en una o varias tablas. Facilita el análisis de datos mediante condiciones, proyecciones y ordenamientos, sin alterar el contenido de la base de datos.

SELECT nombre, edad

FROM clientes;

Consulta con condición:

SELECT * FROM clientes

WHERE edad > 30;

3.2. Operadores de comparación, aritméticos y lógicos

El uso de operadores en SQL resulta fundamental para establecer criterios de búsqueda, comparar valores y realizar operaciones matemáticas dentro de las consultas. Estos elementos amplían las capacidades del lenguaje, permitiendo trabajar con los datos de forma más precisa y acorde a las necesidades de análisis y gestión de la información.

A continuación, se presentan los principales operadores que son los de comparación, lógicos y aritméticos, junto con su función y ejemplos prácticos que ilustran cómo se integran en las consultas para construir condiciones y cálculos dentro de una base de datos:

A. Operadores de comparación

Estos operadores se utilizan para comparar valores y establecer condiciones dentro de una consulta, determinando qué registros cumplen los criterios definidos.

= (igual): verifica si dos valores son exactamente iguales. Se usa para filtrar registros que coinciden con un valor específico.

<> (diferente): evalúa si dos valores no son iguales, permitiendo excluir registros con un valor determinado.

> (mayor que): comprueba si un valor es superior a otro, útil para rangos numéricos o fechas.

< (menor que): determina si un valor es inferior a otro, facilitando la selección de datos por límites mínimos.

>= (mayor o igual): verifica si un valor es mayor o igual a otro, combinando igualdad y superioridad.

<= (menor o igual): evalúa si un valor es menor o igual a otro, integrando igualdad e inferioridad.

```
SELECT * FROM clientes
WHERE edad >= 18;
```

B. Operadores lógicos

Los operadores lógicos permiten combinar o negar condiciones, haciendo posible construir filtros más complejos dentro de las consultas.

AND: exige que todas las condiciones unidas se cumplan simultáneamente.

OR: permite que se cumpla al menos una de las condiciones establecidas.

NOT: niega una condición, excluyendo los registros que la cumplen.

```
SELECT * FROM clientes
WHERE edad > 25 AND nombre = 'Carlos';
```

C. Operadores aritméticos

Estos operadores se emplean para realizar cálculos matemáticos sobre valores numéricos almacenados en las tablas.

+ (suma): permite adicionar valores.

- (resta): permite sustraer un valor de otro.

*** (multiplicación):** permite multiplicar valores, comúnmente usado para cálculos de porcentajes o ajustes.

/ (división): permite dividir un valor entre otro para obtener proporciones o promedios.

```
SELECT salario, salario * 1.1 AS aumento
FROM empleados;
```

Nota: la combinación de operadores permite construir consultas avanzadas y precisas.

3.3. Funciones de cadena, matemáticas, fecha-hora y del sistema

Las funciones en SQL amplían las capacidades de las consultas al permitir transformar, calcular y analizar los datos directamente dentro de la base de datos. Su uso resulta fundamental para optimizar el tratamiento de la información y obtener resultados más precisos sin necesidad de procesamiento externo.

A continuación, se presentan las principales funciones, junto con su propósito y ejemplos prácticos que ilustran su aplicación en consultas habituales:

A. Funciones de cadena

Estas funciones se utilizan para manipular y transformar datos de tipo texto, facilitando su presentación, análisis y validación dentro de las consultas.

- UPPER(): convierte el texto a mayúsculas, útil para estandarizar la información.
- LOWER(): convierte el texto a minúsculas, facilitando comparaciones sin distinción entre mayúsculas y minúsculas.
- LENGTH(): devuelve la cantidad de caracteres de una cadena, permitiendo validar la longitud de los textos.
- SUBSTRING(): extrae una parte específica de una cadena de texto según la posición y longitud indicadas.

```
SELECT UPPER(nombre) FROM clientes;
```

B. Funciones matemáticas

Permiten realizar cálculos numéricos directamente sobre los datos almacenados en la base de datos.

- ROUND(): redondea un número al número de decimales especificado.
- ABS(): retorna el valor absoluto de un número, eliminando el signo negativo.
- SQRT(): calcula la raíz cuadrada de un valor numérico.

```
SELECT ROUND(123.456, 2);
```

C. Funciones de fecha y hora

Se emplean para obtener, manipular y analizar valores temporales, como fechas y horas.

- NOW(): devuelve la fecha y hora actual del sistema.
- CURDATE(): retorna únicamente la fecha actual.
- YEAR(): extrae el año de una fecha determinada.

```
SELECT YEAR(fecha_nacimiento) FROM clientes;
```

D. Funciones del sistema

Proporcionan información del sistema o permiten realizar cálculos agregados sobre los datos.

- COUNT()
- USER()
- VERSION()

SELECT COUNT(*) FROM clientes;

Las funciones permiten optimizar consultas sin necesidad de procesar datos externamente.

3.4. Consultas combinadas (JOIN)

Las consultas combinadas en SQL permiten relacionar información almacenada en diferentes tablas, aprovechando las claves y relaciones definidas en la base de datos. Este tipo de consultas resulta esencial para integrar datos dispersos y obtener resultados coherentes que representen la realidad del sistema de información.

A continuación, se presentan los principales tipos de JOIN, junto con su función y ejemplos prácticos que ilustran cómo se utilizan para combinar registros de múltiples tablas dentro de una consulta:

A. INNER JOIN

Devuelve únicamente los registros que tienen coincidencia en ambas tablas relacionadas, según la condición establecida. Se utiliza cuando se requiere información que exista simultáneamente en las dos tablas.

SELECT c.nombre, p.pedido

FROM clientes c

INNER JOIN pedidos p ON c.id = p.cliente_id;

B. LEFT JOIN

Devuelve todos los registros de la tabla ubicada a la izquierda de la consulta y solo los registros coincidentes de la tabla derecha. Cuando no hay coincidencia, los campos de la tabla derecha se muestran con valores nulos.

```
SELECT c.nombre, p.pedido
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id;
```

C. RIGHT JOIN

Devuelve todos los registros de la tabla ubicada a la derecha de la consulta y únicamente los registros coincidentes de la tabla izquierda. Los campos sin correspondencia se completan con valores nulos.

```
SELECT *
FROM clientes
RIGHT JOIN pedidos;
```

D. FULL OUTER JOIN

Devuelve todos los registros de ambas tablas, tanto los coincidentes como los no coincidentes. Se utiliza cuando se requiere una visión completa de la información contenida en ambas tablas.

```
SELECT *
FROM clientes
```

FULL OUTER JOIN pedidos;

E. CROSS JOIN

Genera todas las combinaciones posibles entre los registros de las tablas involucradas, produciendo un producto cartesiano. Su uso debe ser controlado debido al alto volumen de resultados que puede generar.

SELECT * FROM clientes

CROSS JOIN productos;

El uso de JOIN es clave en sistemas empresariales donde la información está distribuida en múltiples tablas.

3.5. Sentencias de agregación

Las sentencias de agregación en SQL permiten resumir y analizar grandes volúmenes de datos, transformando múltiples registros individuales en información consolidada. Estas operaciones son fundamentales en contextos donde se requiere obtener promedios, totales, conteos o valores extremos para apoyar procesos de análisis y toma de decisiones.

A través de las funciones de agregación y de cláusulas, es posible organizar, filtrar y ordenar los resultados de manera estructurada. Estas sentencias no alteran los datos originales, sino que facilitan su interpretación mediante consultas que agrupan y sintetizan la información almacenada en las tablas. Dichas sentencias se dividen en:

A. Funciones básicas

Las funciones básicas de agregación operan sobre un conjunto de registros y devuelven un único valor como resultado, calculado a partir de los datos contenidos en una columna específica.

- MAX(): devuelve el valor más alto de una columna.
- MIN(): devuelve el valor más bajo de una columna.
- AVG(): calcula el promedio de los valores numéricos.
- SUM(): suma los valores de una columna numérica.
- COUNT(): cuenta la cantidad de registros.

```
SELECT AVG(salario) FROM empleados;
```

B. GROUP BY

La cláusula GROUP BY permite agrupar registros que comparten un mismo valor en una o más columnas, de modo que las funciones de agregación se apliquen a cada grupo y no al conjunto total de datos.

```
SELECT departamento, COUNT(*)

FROM empleados

GROUP BY departamento;
```

C. HAVING

La cláusula HAVING se utiliza para filtrar los resultados después de aplicar una agrupación, es decir, actúa sobre los grupos generados por GROUP BY y no sobre los registros individuales.

```
SELECT departamento, COUNT(*)
FROM empleados
GROUP BY departamento
HAVING COUNT(*) > 5;
```

D. ORDER BY

La cláusula ORDER BY permite ordenar los resultados finales de una consulta en forma ascendente o descendente, facilitando la lectura y el análisis de la información obtenida.

```
SELECT * FROM empleados
ORDER BY salario DESC;
```

Importante:

WHERE filtra registros.

HAVING filtra agrupaciones.

La manipulación y consulta de datos mediante SQL representa el núcleo operativo de las bases de datos, permitiendo transformar información en conocimiento útil. El dominio de estas herramientas facilita la construcción de consultas eficientes, el análisis de datos y la toma de decisiones basadas en información confiable.

Basado en lo anterior, se presenta el siguiente diagrama que resume todo el proceso de ejecución de consulta:

Figura 3. Proceso de manipulación y consulta de datos en SQL



Proceso de manipulación y consulta de datos en SQL

- Tabla de datos
- SELECT
- WHERE (filtro)
- JOIN (combinación de tablas)
- GROUP BY (agrupación)
- Funciones (SUM, AVG, MAX)
- ORDER BY (ordenamiento)
- Resultados

4. Gestión de bases de datos NoSQL

La gestión de bases de datos NoSQL representa una evolución en la forma de almacenar y procesar información, especialmente en entornos donde los datos crecen de manera exponencial y presentan estructuras no uniformes. A diferencia de los sistemas relacionales tradicionales, las bases de datos NoSQL están diseñadas para ofrecer alta escalabilidad, flexibilidad y rendimiento en escenarios de gran volumen de información.

El término NoSQL (Not Only SQL) no implica la ausencia del lenguaje SQL, sino una ampliación del paradigma de gestión de datos, permitiendo el uso de modelos alternativos que se adaptan mejor a aplicaciones modernas como redes sociales, sistemas de análisis en tiempo real y plataformas de comercio electrónico.

Las bases de datos NoSQL no reemplazan a las relacionales, sino que las complementan según el tipo de aplicación.

4.1. Conceptos de gestores NoSQL

Los gestores NoSQL se caracterizan por manejar datos sin necesidad de esquemas rígidos, lo que permite una mayor flexibilidad en el almacenamiento de información. Esta característica es especialmente útil en aplicaciones donde los datos cambian constantemente o no tienen una estructura definida.

Los sistemas NoSQL se clasifican según su modelo de datos de la siguiente manera:

A. Bases de datos orientadas a documentos

Almacenan la información en documentos estructurados, generalmente en formatos como JSON o BSON, lo que permite manejar datos semiestructurados con gran flexibilidad y sin esquemas rígidos.

{

"nombre": "Carlos",

"edad": 30,

"ciudad": "Bogotá"

}

- Alta flexibilidad.
- Fácil integración con aplicaciones web.

B. Bases de datos clave-valor

Guardan la información como pares formados por una clave única y un valor asociado, lo que facilita accesos muy rápidos y resulta ideal para escenarios de alta velocidad y escalabilidad.

usuario_1 → "Camila"

- Alta velocidad.
- Ideal para almacenamiento en caché.

C. Bases de datos columnares

Organizan los datos por columnas en lugar de filas, optimizando la lectura y el análisis de grandes volúmenes de información, especialmente en sistemas de inteligencia de negocios y analítica.

columna_edad → [25, 30, 40]

- Alto rendimiento en análisis de datos.
- Utilizadas en big data.

D. Bases de datos de grafos

Representan los datos mediante nodos, relaciones y propiedades, permitiendo modelar y consultar de forma eficiente relaciones complejas entre entidades.

Persona_A —(AMIGO_DE)—> Persona_B

- Ideales para redes sociales.
- Permiten análisis de relaciones complejas.

Cada modelo responde a necesidades específicas, por lo que su elección depende del tipo de aplicación.

4.2. Instalación y configuración de bases de datos NoSQL

La instalación y configuración de una base de datos NoSQL requiere seguir una secuencia lógica que garantice su correcto funcionamiento y seguridad. Aunque el proceso varía según el motor utilizado, la mayoría de gestores comparten etapas comunes. A continuación, se presenta un paso a paso general, tomando como referencia gestores ampliamente usados como MongoDB:

- **Paso 1. Selección del gestor NoSQL**

Revisar las características del sistema a implementar e identificar el gestor NoSQL más adecuado según el tipo de datos, los requerimientos de escalabilidad y el contexto de uso. Considerar opciones ampliamente adoptadas, como MongoDB, por su flexibilidad y facilidad de administración.

- **Paso 2. Descarga del software**

Acceder al sitio oficial del gestor NoSQL seleccionado y descargar la versión compatible con el sistema operativo. Verificar que se trate de una versión estable y con soporte vigente.

- **Paso 3. Ejecución del instalador**

Ejecutar el instalador siguiendo las instrucciones proporcionadas por el asistente. Durante esta etapa se instalan los componentes básicos y los servicios necesarios para el funcionamiento del sistema.

- **Paso 4. Configuración de rutas y servicios**

Configurar las rutas de almacenamiento de datos y los archivos del sistema. Definir si el gestor se ejecutará como un servicio del sistema para permitir su inicio automático.

- **Paso 5. Inicialización del servidor**

Iniciar el servidor NoSQL y comprobar que se encuentre en ejecución. Verificar que el sistema esté disponible para aceptar conexiones.

- **Paso 6. Configuración básica del sistema**

Definir los parámetros iniciales del gestor, tales como:

  - Directorio de almacenamiento de datos.
  - Puertos de conexión.
  - Usuarios y mecanismos de autenticación.
  - Políticas de acceso según perfiles.

- **Paso 7. Ajustes de seguridad**

Configurar mecanismos de seguridad, como autenticación y cifrado, especialmente en entornos productivos, con el fin de proteger la información y prevenir accesos no autorizados.

Una correcta instalación y configuración de bases de datos NoSQL contribuye a garantizar estabilidad, rendimiento y seguridad en los sistemas de información.

4.3. Construcción de bases de datos NoSQL

A continuación, se presentan las etapas fundamentales en la construcción de una base de datos NoSQL. Cada una de ellas cumple una función específica dentro del proceso, desde la creación inicial de la base de datos hasta la organización de la información y la inserción de los documentos que la conforman, permitiendo una gestión flexible y eficiente de los datos:

A. Creación de base de datos

En gestores NoSQL como MongoDB, la base de datos no requiere una creación explícita previa. Esta se genera automáticamente en el momento en que se insertan los

primeros documentos, lo que simplifica el proceso inicial y permite una implementación más ágil.

use tienda

B. Creación de colecciones

Las colecciones funcionan como contenedores de documentos relacionados. Aunque pueden crearse de forma implícita al insertar información, también es posible definirlas previamente para organizar los datos de manera lógica y facilitar su administración.

db.createCollection("clientes")

C. Inserción de documentos

La inserción de documentos consiste en almacenar la información dentro de las colecciones en formato flexible, como JSON o BSON. Cada documento puede contener distintos campos, lo que permite manejar datos heterogéneos sin necesidad de esquemas rígidos.

db.clientes.insertOne({

nombre: "Ana",

edad: 28,

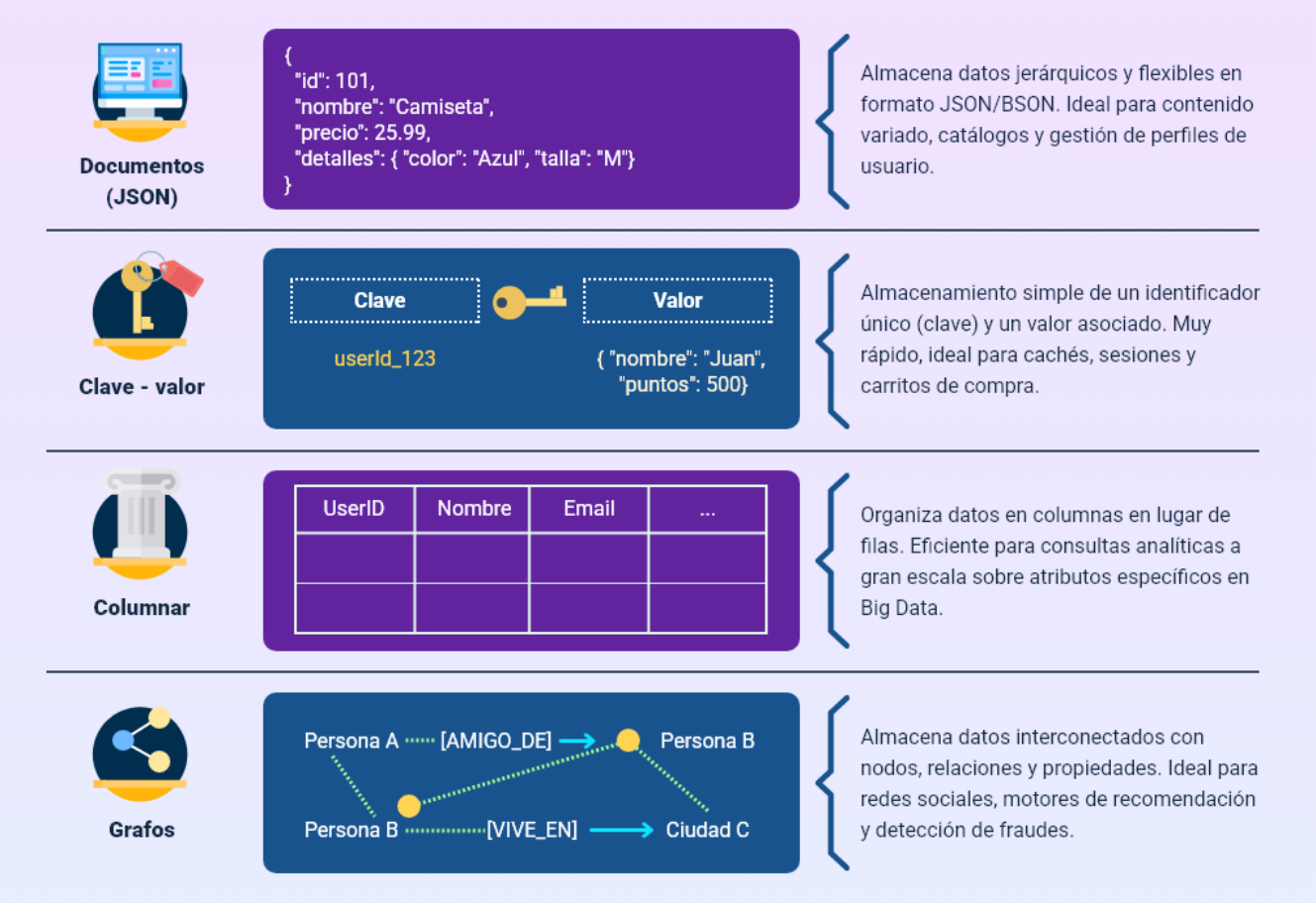
ciudad: "Medellín"

})

La falta de esquema puede generar desorden si no se gestiona adecuadamente.

Para finalizar lo relacionado con los modelos de bases de datos NoSQL, se presenta la siguiente imagen comparativa:

Figura 4. Modelos y estructuras de bases de datos NoSQL



Modelos y estructuras de bases de datos NoSQL

- **Documentos (JSON)**

Almacena datos jerárquicos y flexibles en formato JSON/BSON. Ideal para contenido variado, catálogos y gestión de perfiles de usuario.

Ejemplo:

```
{
  "id": 101,
  "nombre": "Camiseta",
  "precio": 25.99,
  "detalles": {
    "color": "Azul",
    "talla": "M"
  }
}
```

- **Clave-valor**

Almacenamiento simple de un identificador único (clave) y un valor asociado. Muy rápido, ideal para cachés, sesiones y carritos de compra.

Ejemplo:

```
Clave: userId_123
Valor: {
  "nombre": "Juan",
  "puntos": 500
}
```

- **Columnar**

Organiza datos en columnas en lugar de filas. Eficiente para consultas analíticas a gran escala sobre atributos específicos en Big Data.

Ejemplo:

UserID | Nombre | Email | ...

- **Grafos**

Almacena datos interconectados con nodos, relaciones y propiedades. Ideal para redes sociales, motores de recomendación y detección de fraudes.

Ejemplo:

- Persona A —[AMIGO_DE]→ Persona B
- Persona B —[VIVE_EN]→ Ciudad C

4.4. Operaciones CRUD, consultas e índices

Las operaciones CRUD (Create-crear, Read-leer, Update-actualizar, Delete-eliminar) constituyen la base para la administración de la información en bases de datos NoSQL. Estas operaciones permiten crear, consultar, modificar y eliminar datos, y suelen implementarse mediante lenguajes como JavaScript, especialmente en motores orientados a documentos como MongoDB. A continuación, la explicación de cada una:

a) CREATE (Crear - JavaScrip)

Permite agregar nuevos documentos a una colección, almacenando información estructurada sin necesidad de definir previamente un esquema rígido.

```
db.clientes.insertOne({
nombre: "Luis",
edad: 35
})
```

b) READ (Consultar - JavaScrip)

Se utiliza para acceder y visualizar los documentos almacenados, aplicando filtros o criterios para obtener información específica.

```
db.clientes.find({ edad: { $gt: 30 } })
```

c) UPDATE (Actualizar - JavaScrip)

Permite modificar uno o varios campos de documentos existentes, ya sea actualizando valores puntuales o conjuntos de datos completos.

```
db.clientes.updateOne(
{ nombre: "Luis" },
{ $set: { edad: 36 } }
)
```

d) DELETE (Eliminar - JavaScrip)

Se emplea para eliminar documentos de una colección, de forma selectiva o total, según las condiciones definidas.

```
db.clientes.deleteOne({ nombre: "Luis" })
```

Consultas avanzadas:

Las consultas pueden incluir operadores:

- $gt (mayor que).
- $lt (menor que).
- $and.
- $or.

Ejemplo (JavaScrip):

```
db.clientes.find({
$and: [
{ edad: { $gt: 25 } },
{ ciudad: "Bogotá" }
]
})
```

Las bases de datos NoSQL representan una solución moderna y eficiente para el manejo de grandes volúmenes de información no estructurada, ofreciendo flexibilidad, escalabilidad y alto rendimiento. Su correcta implementación permite complementar los sistemas tradicionales, adaptándose a las necesidades de aplicaciones actuales que requieren procesamiento dinámico y distribuido de datos.

5. Seguridad, pruebas y respaldo de bases de datos

La seguridad, las pruebas y el respaldo de bases de datos constituyen componentes críticos en la administración de la información, ya que garantizan la protección, confiabilidad y disponibilidad de los datos dentro de un sistema. En un entorno donde la información es uno de los activos más valiosos, la implementación de estrategias adecuadas en estos aspectos resulta indispensable para prevenir pérdidas, accesos no autorizados y fallos en el sistema.

La gestión adecuada de la seguridad permite controlar quién accede a los datos y qué acciones puede realizar, mientras que las pruebas aseguran la correcta estructura y funcionamiento de la base de datos. Por su parte, los mecanismos de respaldo permiten recuperar la información ante fallos, errores humanos o ataques informáticos.

La ausencia de políticas de seguridad y respaldo puede generar pérdidas irreversibles de información.

5.1. Lenguaje de control de datos (DCL): GRANT y REVOKE

La gestión de usuarios y permisos es un componente esencial de la seguridad en las bases de datos, ya que permite controlar quién puede acceder a la información y qué acciones puede realizar dentro del sistema. Mediante estos mecanismos se protege la integridad, confidencialidad y disponibilidad de los datos, especialmente en entornos multiusuario.

A continuación, se presentan las principales sentencias utilizadas para administrar permisos en SQL con sus respectivos ejemplos aplicados, las cuales permiten asignar o retirar privilegios sobre objetos de la base de datos de forma controlada y segura:

A. GRANT

La sentencia GRANT se utiliza para otorgar permisos específicos a usuarios o roles, permitiéndoles realizar acciones como consultar, insertar, actualizar o eliminar datos sobre tablas, vistas u otros objetos de la base de datos. Su uso adecuado facilita el acceso controlado según las responsabilidades de cada usuario.

```
GRANT SELECT, INSERT ON clientes TO usuario1;
```

B. REVOKE

La sentencia REVOKE permite retirar permisos previamente concedidos, limitando o eliminando el acceso de un usuario a determinados recursos. Esta instrucción es fundamental para mantener la seguridad cuando cambian los roles, se detectan riesgos o ya no es necesario conservar ciertos privilegios.

```
REVOKE INSERT ON clientes FROM usuario1;
```

De acuerdo a los tipos de permisos se encuentran:

- SELECT: consultar datos.
- INSERT: insertar registros.
- UPDATE: modificar datos.
- DELETE: eliminar registros.
- ALL PRIVILEGES: acceso total.

Un mal manejo de permisos puede comprometer la seguridad de toda la base de datos.

5.2. Pruebas de bases de datos: scripts de definición, integridad de datos e integridad referencial

Las pruebas de bases de datos constituyen una etapa fundamental para asegurar el correcto funcionamiento de los sistemas de información, ya que permiten verificar que la estructura, los datos y las relaciones definidas respondan a los requerimientos funcionales y normativos del sistema. A través de estas pruebas se minimizan errores, se previenen inconsistencias y se garantiza la confiabilidad de la información almacenada.

A continuación, se abordarán los principales tipos de pruebas aplicadas a bases de datos, enfocadas en la validación de la definición de estructuras, la calidad de los datos y la coherencia de las relaciones, proporcionando una visión integral del control y aseguramiento de la información:

A. Pruebas de scripts de definición

Estas pruebas se centran en verificar que los scripts utilizados para crear y modificar la base de datos se ejecuten correctamente y sin errores. Evalúan la correcta definición de tablas, campos, tipos de datos, claves primarias, claves foráneas y restricciones, asegurando que la estructura de la base de datos se ajuste al diseño establecido.

Acciones:

- Verifica estructura.

- Evalúa restricciones.

B. Pruebas de integridad de datos

Este tipo de pruebas valida que la información almacenada cumpla con las reglas de negocio y restricciones definidas, como valores obligatorios, rangos permitidos y formatos correctos. Su objetivo es evitar el ingreso de datos erróneos o inconsistentes que puedan afectar los procesos y la toma de decisiones.

Acciones:

- No permitir valores nulos en campos obligatorios.
- Evitar duplicados.

C. Pruebas de integridad referencial

Estas pruebas garantizan la coherencia entre las relaciones de las tablas, verificando que las claves foráneas correspondan a registros válidos en las tablas relacionadas. De esta manera, se previene la existencia de registros huérfanos y se asegura la consistencia lógica de la información dentro del sistema.

Acciones:

- No permitir registros huérfanos.
- Mantener consistencia entre tablas relacionadas.

Estas pruebas son fundamentales antes de pasar a ambientes productivos.

5.3. Gestión de copias de seguridad: exportación e importación de bases de datos

La gestión de copias de seguridad constituye un componente esencial en la administración de bases de datos, ya que permite proteger la información frente a fallos técnicos, errores humanos o incidentes de seguridad. Mediante la realización de respaldos adecuados se garantiza la continuidad del servicio y la recuperación de los datos en caso de pérdida o corrupción.

Una estrategia efectiva de copias de seguridad combina distintos tipos de respaldo, como copias completas, incrementales y diferenciales, con el fin de optimizar el equilibrio entre seguridad, consumo de recursos y tiempo de recuperación. Esta planificación resulta clave para asegurar la disponibilidad y confiabilidad de la información en entornos operativos y productivos.

A continuación, se describen y ejemplifican dos procesos fundamentales asociados a la gestión de copias de seguridad, que permiten respaldar y restaurar bases de datos de forma controlada y segura:

A. Exportación de bases de datos

La exportación consiste en generar archivos que contienen la estructura y/o los datos de una base de datos, los cuales pueden almacenarse en medios externos o ubicaciones seguras. Este proceso facilita la creación de respaldos, la migración de información entre sistemas y la transferencia de datos entre distintos entornos, como desarrollo, pruebas y producción.

</> Bash

```
mysqldump -u root -p base_datos > respaldo.sql
```

B. Importación de bases de datos

La importación permite restaurar la información previamente exportada, incorporándola nuevamente en el sistema de base de datos. Este procedimiento es fundamental para la recuperación ante fallos, la reconstrucción de entornos y la continuidad operativa, asegurando que los datos se restablezcan de manera íntegra y coherente.

</> Bash

```
mysql -u root -p base_datos < respaldo.sql
```

- **Buenas prácticas**

La aplicación de buenas prácticas en la gestión de copias de seguridad es fundamental para garantizar la protección, disponibilidad y recuperación oportuna de la información. Estas prácticas permiten reducir riesgos operativos, prevenir pérdidas de datos y asegurar que los respaldos sean confiables y funcionales cuando se requieran. A continuación, se presentan algunas recomendaciones esenciales que fortalecen la administración adecuada de los procesos de backup:

- Realizar copias periódicas.
- Almacenar respaldos en ubicaciones externas.
- Verificar la integridad de los respaldos.
- Automatizar procesos de backup.

La implementación de mecanismos de seguridad, pruebas y respaldo en bases de datos es fundamental para garantizar la integridad, disponibilidad y confidencialidad de la información. Estas prácticas no solo previenen pérdidas de datos, sino que también

fortalecen la confiabilidad de los sistemas y permiten una recuperación efectiva ante cualquier eventualidad.

Basado en lo anterior y para finalizar lo relacionado con la seguridad de las bases de datos, se presenta esta imagen:

Figura 5. Ciclo de seguridad, pruebas y respaldo de bases de datos



Ciclo de seguridad, pruebas y respaldo de bases de datos

- Control de acceso: (GRANT / REVOKE)
- Validación de datos
- Pruebas
- Respaldo: (Backup)
- Recuperación: (Restore)

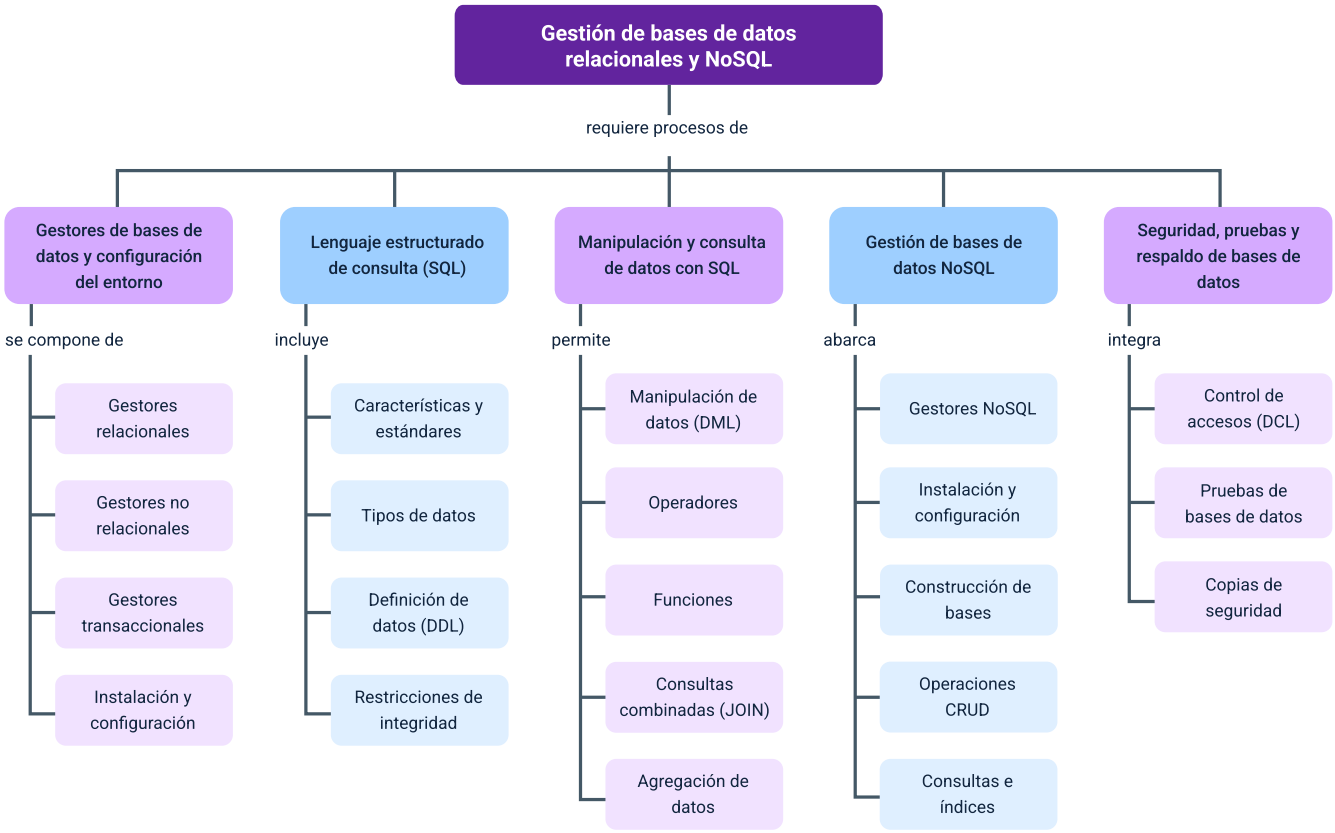
Síntesis

El estudio de los gestores de bases de datos, el lenguaje SQL, la manipulación de datos, las tecnologías NoSQL y las estrategias de seguridad permite comprender de manera integral el ciclo de vida de la información dentro de los sistemas informáticos.

A lo largo del desarrollo del contenido, se abordaron aspectos fundamentales que van desde la configuración del entorno hasta la administración avanzada de datos, integrando conocimientos técnicos y prácticos necesarios para el diseño e implementación de soluciones eficientes.

Asimismo, se destacó la importancia de seleccionar adecuadamente las tecnologías según el contexto, garantizando un equilibrio entre rendimiento, escalabilidad y seguridad. La integración de bases de datos relacionales y NoSQL permite responder a las demandas actuales del entorno digital, donde la información crece constantemente y requiere ser gestionada de manera eficiente.

Finalmente, la aplicación de buenas prácticas en seguridad, pruebas y respaldo consolida la confiabilidad de los sistemas, asegurando la protección de la información y la continuidad operativa. Este enfoque integral permite formar profesionales capaces de administrar datos de manera estratégica, contribuyendo al desarrollo de soluciones tecnológicas robustas y alineadas con las necesidades del entorno actual.


Gestión de bases de datos relacionales y NoSQL
requiere procesos de
Gestores de bases de datos y configuración del entorno
se compone de
Gestores relacionales
Gestores no relacionales
Gestores transaccionales
Instalación y configuración
Lenguaje estructurado de consulta (SQL)
incluye
Características y estándares
Tipos de datos
Definición de datos (DDL)
Restricciones de integridad
Manipulación y consulta de datos con SQL
permite
Manipulación de datos (DML)
Operadores
Funciones
Consultas combinadas (JOIN)
Agregación de datos
Gestión de bases de datos NoSQL
abarca
Gestores NoSQL
Instalación y configuración
Construcción de bases
Operaciones CRUD
Consultas e índices
Seguridad, pruebas y respaldo de bases de datos
integra
Control de accesos (DCL)
Pruebas de bases de datos
Copias de seguridad

Glosario

Backup: copia de seguridad de la información que permite su recuperación en caso de pérdida o fallo del sistema.

Base de datos: conjunto organizado de información que se almacena y gestiona de manera estructurada para facilitar su acceso y manipulación.

Clave foránea: campo que establece una relación entre tablas dentro de una base de datos.

Clave primaria: campo que identifica de manera única cada registro dentro de una tabla.

DBMS: sistema de gestión de bases de datos que permite crear, administrar y consultar información de manera eficiente.

DDL: lenguaje de definición de datos utilizado para crear y modificar la estructura de una base de datos.

DML: lenguaje de manipulación de datos que permite insertar, actualizar, eliminar y consultar información.

Índice: estructura que mejora la velocidad de búsqueda y recuperación de datos en una base de datos.

NoSQL: tipo de base de datos que permite almacenar datos no estructurados o semiestructurados con alta escalabilidad.

SQL: lenguaje estructurado de consulta utilizado para gestionar y manipular bases de datos relacionales.

Referencias bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

GeeksforGeeks. (s.f.). Database Management System Tutorial. https://www.geeksforgeeks.org/dbms/

IBM. (s.f.). What is NoSQL? https://www.ibm.com/topics/nosql-databases

Microsoft. (s.f.). SQL Server Documentation. https://learn.microsoft.com/sql/

MongoDB Inc. (s.f.). MongoDB Manual. https://www.mongodb.com/docs/

MySQL. (s.f.). MySQL Reference Manual. https://dev.mysql.com/doc/

Oracle. (s.f.). Oracle Database Documentation. https://docs.oracle.com/en/database/oracle/oracle-database/

PostgreSQL Global Development Group. (s.f.). PostgreSQL Documentation. https://www.postgresql.org/docs/

W3Schools. (s.f.). SQL Tutorial. https://www.w3schools.com/sql/

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Experto temático	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador `full stack`	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador `full stack`	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima